

ENTERPRISE TECHNICAL BLUEPRINT

Full-Stack Architecture, Admin Portal & OCI Deployment

Comprehensive guide for transitioning the StyleX Vite frontend into a live transactional salon platform with real-time slot reservation, admin controls, customer marketing CRM, and single-instance Oracle Cloud Infrastructure (OCI) hosting.

1. Executive System Overview

The StyleX application currently operates as a high-performance, client-side Single Page Application (SPA). To support **real-time booking confirmation**, **slot freezing**, **promotional image management**, and **customer CRM data capture**, the frontend connects to a decoupled Node.js / PostgreSQL backend mediated by Nginx.

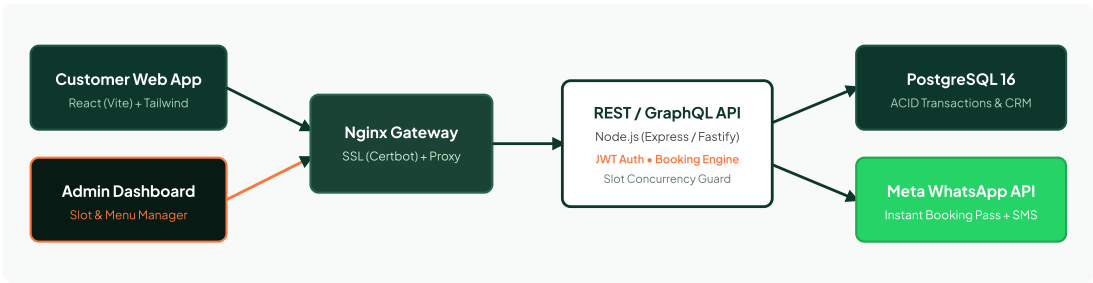


FIGURE 1.0: END-TO-END SYSTEM DATA FLOW & MICRO-SERVICE ARCHITECTURE

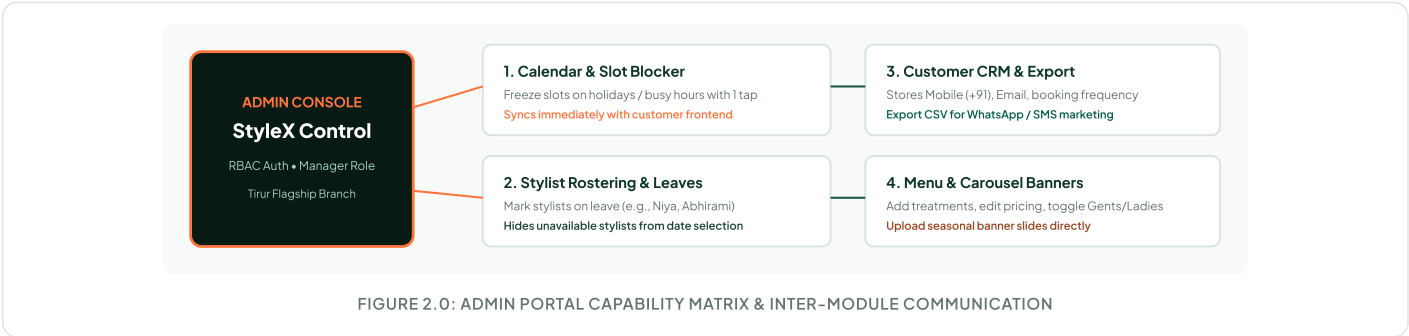
- ### A. Headless CMS Approach (Fastest)

Using **Strapi** or **Directus** connects automatically to PostgreSQL and provides a pre-built admin panel for editing services, banners, and prices. Best if speed to market is top priority.
- ### B. Custom Node.js API (Most Scalable)

Using **Express / NestJS + Prisma ORM**. Delivers 100% control over slot-concurrency algorithms, WhatsApp webhooks, and custom reporting tailored strictly to StyleX operations.

2. Admin Dashboard Functional Modules

The administrative suite requires 5 distinct modules accessible only by authenticated salon managers. Here is the operational breakdown of each module:



Slot Blocking & VIP Overrides

Allows reception desk to freeze specific hours (e.g. 5:00 PM – 7:00 PM on Friday) for walk-in surges or private bridal party bookings. The customer frontend will automatically grey out those slots in real-time.

Promotional Media Uploader

Replaces hardcoded entries in `PROMO_SLIDES` with a drag-and-drop uploader. Automatically resizes images, generates responsive WebP assets, and allows setting active start/end dates.

3. Database Entity Relationship Model (PostgreSQL)

A relational schema guarantees ACID transactions to prevent double-booking identical slots:

Table Name	Primary Key	Key Fields & Types	Description
customers	id (UUID)	phone (VARCHAR), country_code, email, full_name, created_at	Customer CRM data for SMS and WhatsApp campaigns
services	id (VARCHAR)	name, category, gender ('gents' 'ladies'), duration_mins, price	Tirur flagship menu items & treatments
stylists	id (VARCHAR)	name, specialty, gender, is_active (BOOL)	Staff roster (Niya, Abhirami, Saneesh, Sunita, etc.)
bookings	id (UUID)	booking_ref (SX-####), customer_id, service_id, stylist_id, date, time_slot, status	Active and historical appointment ledger
blocked_slots	id (INT)	date (DATE), time_slot (VARCHAR), reason, stylist_id (NULLable)	Overrides and salon closures set by admin

Double-Booking Prevention Logic

When a client submits an appointment, the API initiates a PostgreSQL transaction with a row lock (`SELECT * FROM bookings WHERE date = $1 AND time_slot = $2 AND (stylist_id = $3 OR stylist_id IS NULL) FOR UPDATE`). If an existing confirmed appointment exists, the system immediately returns an error requesting the user select an adjacent time slot.

4. Hosting on a Single OCI Compute Instance

Deploying on a single **12 GB RAM, 200 GB NVMe Storage** OCI Compute Instance (such as the Ampere A1 ARM or AMD E4 Standard) is an outstanding, cost-effective choice. It will effortlessly handle up to **50,000+ daily visitors** and hundreds of concurrent booking transactions.

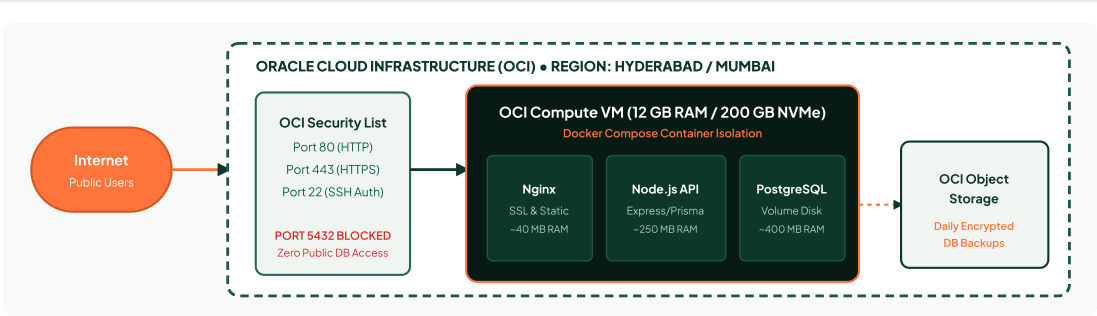


FIGURE 3.0: OCI SINGLE-INSTANCE CLOUD INFRASTRUCTURE & NETWORK SECURITY LAYOUT

Memory & Resource Footprint Breakdown (12 GB Total Capacity)

Component	RAM Allocation	Storage Footprint	Operational Role
Ubuntu / Linux OS	~600 MB	~15 GB	Kernel, systemd, SSH, logging daemon
Nginx Web Server	~50 MB	~100 MB	Serves Vite React production bundle + SSL termination
Node.js Backend (Express)	~250 MB	~300 MB	Booking logic, WhatsApp webhooks, admin authentication
PostgreSQL 16 Database	~500 MB (buffer cache)	~5 GB (up to 5 yrs)	Customers, bookings, audit trail & salon menus
Total Active Usage	~1.4 GB / 12 GB	~20.4 GB / 200 GB	Peak load usage: < 15% capacity (Over 10 GB headroom!)

OCI Storage Longevity

Even with high-resolution promotion images and 100,000+ customer records, 200 GB of NVMe block storage is virtually impossible to fill for several years. You will have over 170 GB of completely free disk space.

5. Critical Production Checklist for OCI

Follow these operational best practices when provisioning your single OCI instance:

1. Host & OS Firewall Configuration

Oracle Linux and Ubuntu on OCI have strict internal `iptables`. Run:

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo netfilter-persistent save
```

2. Free Automated SSL with Certbot

Run Let's Encrypt Certbot to issue and auto-renew SSL certificates every 90 days:

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d stylexsalon.in -d www.stylexsalon.in
```

6. Production Docker Compose Template

Save this as `docker-compose.yml` on your OCI server to run the entire stack in isolated, auto-restarting containers:

```
version: '3.8'

services:
  # 1. Reverse Proxy & Frontend Static Delivery
  nginx:
    image: nginx:alpine
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./dist:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - /etc/letsencrypt:/etc/letsencrypt:ro
    depends_on:
      - backend

  # 2. Node.js REST API
  backend:
    build: ./backend
    restart: always
    environment:
      - DATABASE_URL=postgres://stylex_admin:SuperSecretKey@postgres:5432/stylex_db
      - JWT_SECRET=BespokeTirur2026StyleXKey
      - WHATSAPP_API_TOKEN=EAABwz...
    depends_on:
      - postgres

  # 3. PostgreSQL Database
  postgres:
    image: postgres:16-alpine
    restart: always
    environment:
      - POSTGRES_USER=stylex_admin
      - POSTGRES_PASSWORD=SuperSecretKey
      - POSTGRES_DB=stylex_db
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

7. Automated Nightly Database Backup Cron

Add a cron job (`crontab -e`) to backup the database every night at 2:00 AM (after the salon closes at 1:00 AM) and upload to an OCI Object Storage bucket:

```
0 2 * * * docker exec -t stylex_postgres pg_dump -U stylex_admin stylex_db | gzip > /backups/db_$(date +%F).sql.gz && oci os object put -bn stylex-bac
```

Verdict & Next Steps

Your single 12 GB RAM / 200 GB OCI instance is **100% capable, robust, and cost-effective** for running the complete StyleX client application, admin portal, PostgreSQL database, and automated notifications.